Comp 549: Programming Assignment 2

Bill Zimmerman
260981881

Part 1 – Understanding Helmholtz networks

Question 1.1 (4 marks): In your own words (don't just quote the paper), what is the goal of a Helmholtz machine? In other words, what are we trying to learn with a Helmholtz machine?

The primary goal of a Helmholtz machine is to learn a generative model of observed data—a model that can not only generate realistic data samples but also capture the underlying causes that give rise to those observations. In other words, by training the machine to reproduce optical inputs (such as images or text), it implicitly learns an internal representation of the world that explains these inputs beyond a mere collection of pixels or binary values.

This learning process involves two interconnected components: a generative model that synthesizes realistic data from explanations, and a recognition model that infers these explanations from the observed data. One might think of it as the machine asking, "Given what I know about how such inputs are generated, what is the most likely cause behind this observation?" This mirrors, in a simplified way, how our brains process sensory information—transforming raw inputs like light into meaningful concepts such as a text on a page, a moving car, or a spoken sentence.

Question 1.2 (8 marks): In your own words, what is the description length cost of a representation in a Helmholtz machine? What does a low description length imply for the recognition and generative pathways in a Helmholtz machine? Hint: the answer to this question lies in equations (2) and (3) of the Hinton et al. (1995) paper.

The description length cost of a representation in a Helmholtz machine captures both how well an explanation fits the data and how complex that explanation is. It is defined as:

$$C(\alpha, d) = C(\alpha) + C(d \mid \alpha)$$

The second term, $C(d \mid \alpha)$, is low when the generative model can accurately reconstruct the observed data from the explanation, meaning the explanation fits the data well. The first term, $C(\alpha)$, is low when the recognition model assigns high probability to the explanation, meaning it is a likely or simple explanation, not overly complex or unlikely.

So, a low total description length means the recognition model confidently infers a plausible explanation, and the generative model can reconstruct the data effectively using that cause. In this case, the Helmholtz machine has found a compact and efficient representation of the input.

Question 1.3 (8 marks): What is the loss function that Helmholtz networks reduce? Explain in your own words what the different variables and terms are and what they mean for learning. Hint: the answer to this question lies in equations (5), (6), and (8) of the Hinton et al. (1995) paper and equations (2.2)-(2.6), of the Dayan et al. (1995) paper. Compare the two different papers' equations to help yourself understand what they mean.

Helmholtz networks attempt to reduce this loss function. Taken from Dayan et al. (1995). This loss function represents the log likelihood of seeing the data d given θ , the parameters of the recognition model. In reality, it minimizes the negative log-likliehood.

$$\log p(d \mid \theta) = -\sum_{\alpha} Q_{\alpha} E_{\alpha} - \sum_{\alpha} Q_{\alpha} \log Q_{\alpha} + \sum_{\alpha} Q_{\alpha} \log[Q_{\alpha}/P_{\alpha}] \quad (2.5)$$

$$= -F(d;\theta,Q) + \sum_{\alpha} Q_{\alpha} \log[Q_{\alpha}/P_{\alpha}] \quad (2.6)$$

The first term in 2.6 describes the variational free energy of Q. The free energy is a measure of how well our model explains the data that we have seen from Q, like the cost description we mentioned above. What this means for learning is that we are trying to make the model explain the data better

The second, the KL divergence, which is a measure of the difference between P(θ |d) and Q(θ |d). Q is an approximate distribution of the intractable P that is based on observed data and allows use to have a prior. Ideally, Q should be as close to P as possible, and thus the loss function wants to reduce the KL divergence as much as possible. For learning this means that we are forcing the model to produce outputs similar to the inputs it has seen.

Question 1.4 (4 marks): There are two different phases of training for a Helmholtz machine with different weight updates, a wake phase, and a sleep phase. What are the weight updates for each phase? Define all the variables clearly and describe in your own words how to calculate them. Hint: see equations (4) and (7) of the Hinton et al. (1995) paper.

The wake phase of the Helmholtz machine works by taking input data from the world and propagating it up the layers. The data is passed to ever higher layers of the network via bottom-up recognition weights and the binary activation of the units are computed by sampling their net inputs, if it is above a certain threshold, activate that unit. Once the activity has propagated to the top and an explanation is formed, it is then sent back to update the generative weights. The activity of one layer, or the explanation, is sent back down and the generative weights predict the activations. Those activations are then compared to the real activations that the forward pass produced, and the weights are updated. This is defined by (4) in the Hinton paper.

$$\Delta w_{kj} = \epsilon s_k^{\alpha}(s_j^{\alpha} - p_j^{\alpha}) \quad (4)$$

s_k^α is the actual of the current layer and s_j^α is the actual activity of the layer below. These are generated by the recognition pass. Information is propagated up and the sigmoid of the net inputs is used to compute the activation.p_j^α is the predicted activity of the layer below based on the activity propagated to it. It is calculated by taking the net input of the units, via the generative pathway, and sigmoid is used to convert the value to a probability distribution. To summarize simply, we start with our recognized explanation and go down asking "if we had dreamed this, what input data would we have produced" and then we compare the two.

The sleep phase is similar, it is just in the opposite direction. We produce a dream, and we send it downwards, sampling activities based on the sigmoid of the net inputs. Then at the end we generated data. We pass the data back upwards asking "if we saw this input, what explanation would we have assigned to this" and if the the weights are updated to align this with the actual value.

$$\Delta w_{jk} = \epsilon s_j^\gamma (s_k^\gamma - q_k^\gamma) \qquad (7)$$

Question 1.5 (8 marks): What components of the loss function do each training phase (wake and sleep) help to reduce? In your own words, how do they do this and why are they complementary? Hint: Consider figure 2 from the Dayan et al. (1995) paper.

In plain terms, the wake phase makes sure that generated data resembles real world data, and the sleep phase ensures that the real-world data is adequately explained.

As outlined above, the free energy component of the loss function is an attempted to measure the models's ability to explain the data that we see. The higher it is (or lower it is in the negative log case), the better the model's generated data is able to recreate the expected data. This is what the wake function does. It ensures that the generative pathway is able to recreate the inputs.

The second part of the loss function, the KL divergence, serves as a method of ensuring the approximate distribution Q matches P well. Since Q is used to generate posterior probabilities, it is essential that it generates the correct ones. This is the sleep function's purpose, to ensure that the explanations (posteriors) of some input match the dream that generated them.

These functions are complementary because the outputs of one set of weights are used to update the others. Imagine if one was not used. Say there was no sleep phase, and the wake function was continuously running. It would be updated the generative weights to match incorrect recognition pathways. The same is true in the reverse as the sleep phase would work to update the recognition pathway to match nonsensical generations.

Thinking graphically and considering figure 2 of the Dayan paper, we can see that each step is able to move us in either the X or Y direction. Wake can update the generative params and move us in the Y and sleep can update the recognition distribution and move us along the X. If we

were constrained simply to one of these directions, it would not be possible to find the optimal value.

Part 2 – Filling in the code for the Helmholtz network

Question 2.1 (4 marks): What variables do you need to calculate here to make your weight update? Why are these variables necessary for reducing the various components of the loss function? Hint: Consider your answer to Questions 1.4 and 1.5 here.

In the wake phase I defined the following variables:
upper_activity: the activity of the layer above the one who's generative weights are being minimized
gen_pred: the generative model's prediction of the activity at this layer
actual: the actual activity of the model at that function

In the sleep phase I defined these variables:
lower_activity: the activity of the layer below the one who's recognition weights are being minimized
rec_pred: the recognition model's prediction of the activity at this layer
actual: the actual activity of the model at that function

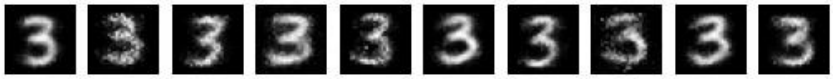
The loss function is defined around reducing the difference between predicted activity to the actual activity. This why it is necessary to define the gen/rec_pred and actual variables as those can store these values. Then it is also necessary to define the upper or lower activities. This is because the generative/recognition predictions are functions of the activities of the upper or lower level and so, by the chain rule, they must be considered.

Question 2.2 (4 marks): Is your code well commented, is it logical? This question is rhetorical, don't answer it. The TAs will mark you based on your code style here. You get all 4 marks only if they can understand your code in 5 minutes flat.

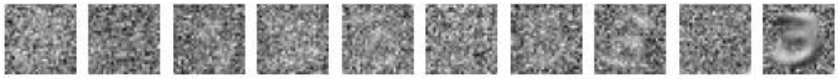
Part 3 – Examining your Helmholtz network

Question 3.1 (5 marks): What happens when you train the network on a single class? What sorts of "dreams" does the network have and what do the weights show? What about if you train it on two classes? And what if you train it on all the classes? Given what you see, do you believe that the network is achieving its ultimate goals in training or not (consider your answer to Question 1.1)? What problems do you see?

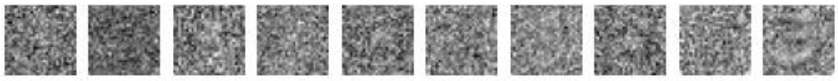
When training on one class, the dreams are incredibly clear, and they basically mimic the inputs. Further, the weights show this kind of overfitting as the right most generative and recognition weights of the top layer simply resemble a 3. Rather than learning the explanation and the factors behind a 3, the model seems to simply be parroting the inputs it has memorized

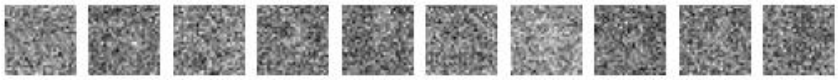


Recognition Weights



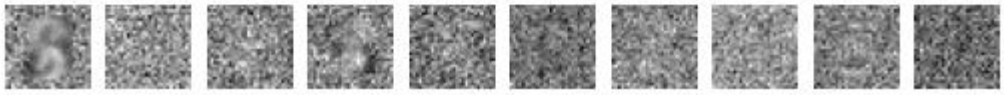
Generative Weights



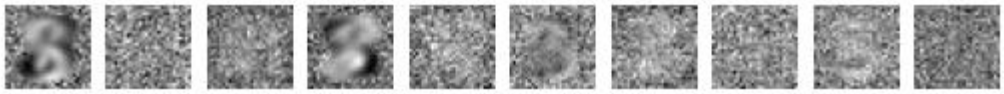


When we train the network on two classes, the results change noticeably. The generated 'dreams' become fuzzier and less like direct copies of individual examples compared to the single-class scenario. Additionally, both the recognition and generative weights begin to show more variability, with features such as vague circular outlines and the characteristic horizontal strokes of eights and threes emerging. Although these features are still not sharply defined, they indicate that the network is starting to learn slight combinations of features that capture aspects of both classes, rather than merely reproducing what it has already seen from one class. However, the learning is not ideal.

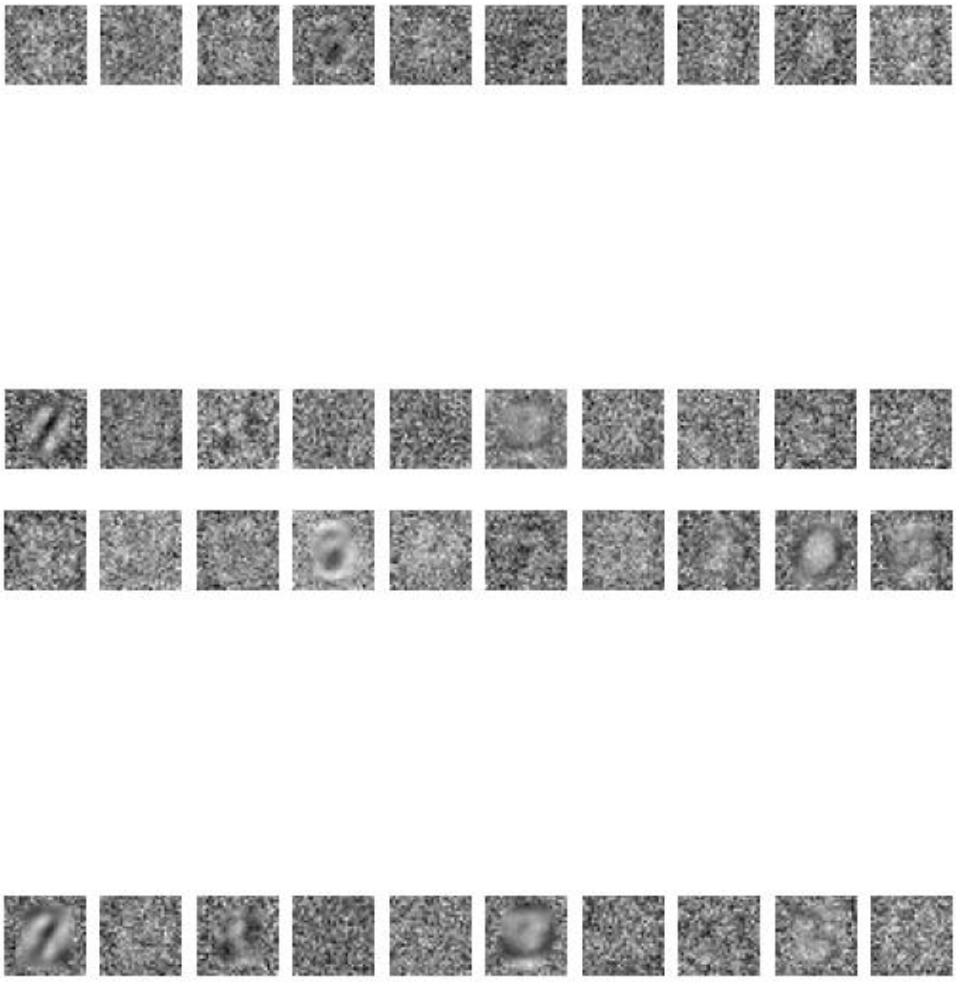
Rec Weights



Gen Weights



Finally, when experimenting with every single class we can see that the model is learning a few different classes. While they seem to be dominated by the classes like 0,7,8,9 The generative weights show more kinds of patterns. However, the learning here is still far from ideal and ultimately I believe that my model is not able to learn.



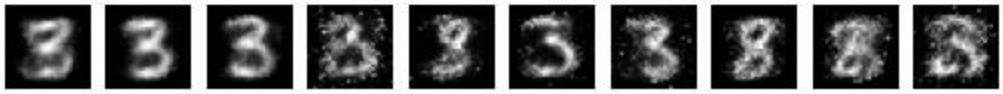
Question 3.2 (5 marks): What happens when you increase the number of hidden units by a lot (e.g. to 100)? Does this change what you said in Question 3.1 at all? Note that you may have to adjust the learning rate when you adjust the number of units.

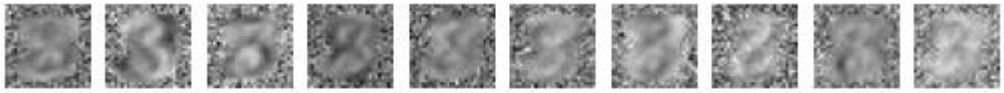
Finally, when going up to 100 units we see immense difficulty to learn. Despite intense hyperparameter tuning the model was not able to learn explanations of the model and the weights continued to just be noise.

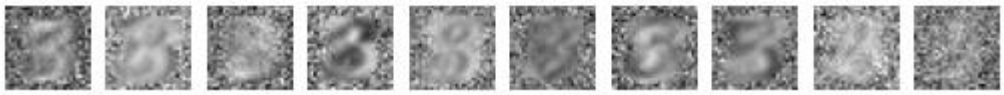
Question 3.3 (10 marks + 5 bonus marks): What happens when you turn off either the wake or sleep phase (one at a time, not both at once, which obliviously just eliminates any training)? How does it affect the dreams and the weights? Explain these results in your own words.

When running just the wake phase, we see that the generated values become much noisier and can lose the general shape that we saw when training on two classes. Since the wake phase trains

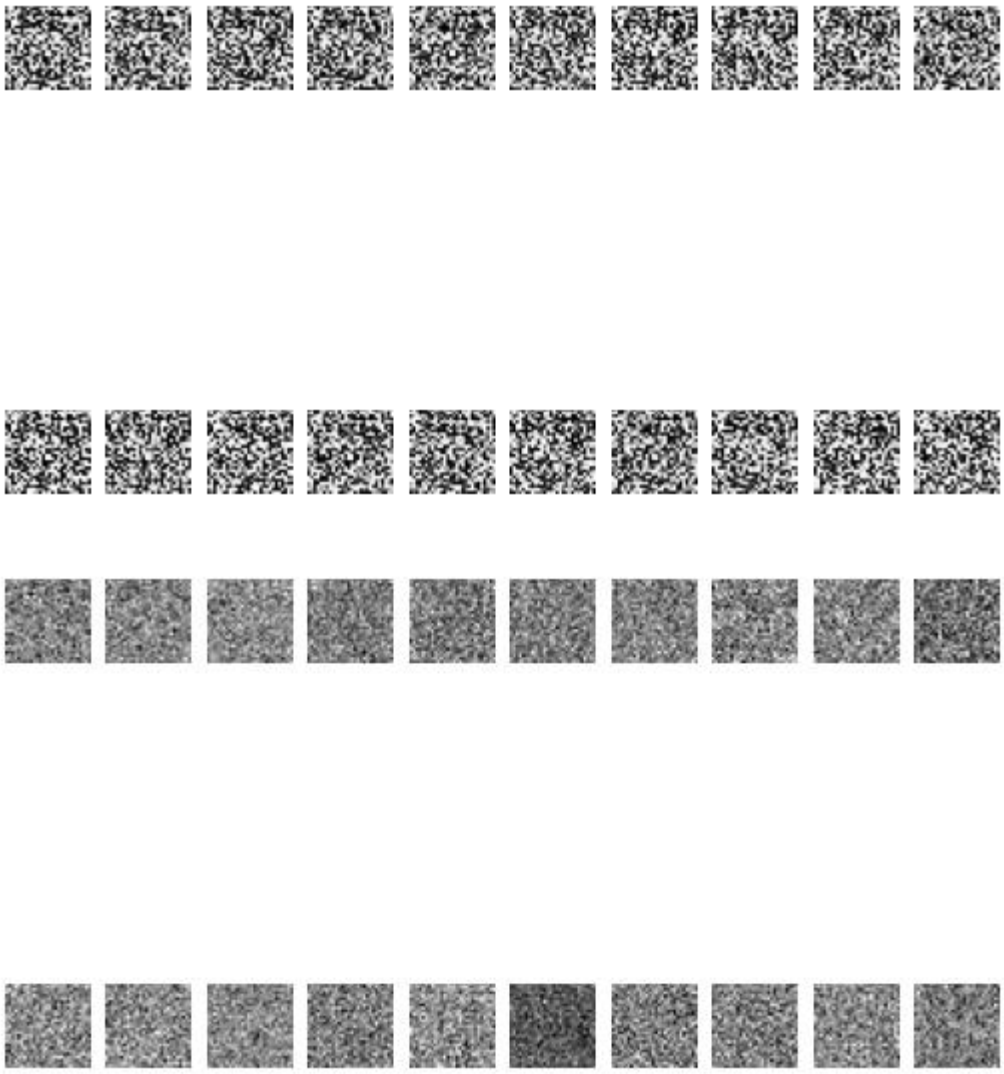
the generative model to at least resemble the inputs we see decent performance but we also see very clearly that the model is simply just learning numbers in the generative weights tend to just represent numbers







When running just sleep we see an almost complete breakdown of the model. Obviously, since the generative weights are not updated the dreams appear to be noise. The generative pathways are never updated and thus, the model's weights are never able to learn any kind of useful recognition. This results in both weights just appearing to be noise



For 5 bonus marks: How does turning off either wake or sleep affect the description length cost? Can you explain why using the loss function and your answer to Question 1.4, and can you use this to explain why the description length starts to increase a bit during training?

When the wake phase is turned off, the generative model is not updated to better reconstruct the data, so the reconstruction error remains high; in other words, the term in the loss function that measures how well the model explains the data, the conditional cost C(d|α), does not decrease, leading to a higher overall description length.

On the other hand, if we turn off the sleep phase, the recognition network is not adjusted to infer an accurate explanation, which means that the KL divergence between Q and P, $C(\alpha)$, stays high. In both cases, the description length remains elevated because one part of the model cannot effectively learn its role.

So how does this explain why the cost length increases during training? In simple terms, early in training both the reconstruction error and the cost of encoding the explanations variables drop. However, as training continues the network begins to capture finer details in the data. This means it starts to represent more information in its explanation, which makes that code more complex and “expensive” to encode (reflected in a higher KL divergence). Even though the reconstruction gets better, the additional complexity in the explanation representation causes the overall description length to increase a bit.